

COMP2200/COMP6200 Practical Exercise (Week 8): Matplotlib Line Plots and Data Wrangling

School of Computing

Preparation

Choose your environment: either run locally with uv or use Google Colab.

1. Local with uv:

- (a) Install uv.

MacOS/Linux: `curl -LsSf https://astral.sh/uv/install.sh | sh`

Windows: `powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"`

- (b) Create a folder for this practical (for example, `week8-prac`) and open a terminal in it.

- (c) Set up your environment:

```
uv init
uv add pandas numpy matplotlib scikit-learn
```

- (d) Launch Jupyter with `uv run --with jupyter jupyter lab` (or `uv run --with jupyter jupyter notebook`).

2. Google Colab:

- (a) Open colab.research.google.com and create a new notebook.
- (b) Upload `flight-on-time-performance.csv` from iLearn or this week's zip file.
- (c) Install packages with `!pip install pandas numpy matplotlib scikit-learn`.

Part A — Exploring Australian Flight Data (50 min)

In this practical, we'll work with Australian flight on-time performance data spanning 2004–2024. You'll practice the Matplotlib workflow from lectures 8A–8C, focusing on line plots and data preprocessing.

A1. Load and explore the data

Launch your notebook and load the flight data.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
flights = pd.read_csv("flight-on-time-performance.csv")
```

1. Examine the data structure: `flights.head()`, `flights.info()`, `flights.columns`.
2. What columns are available? What time period does the data cover?
3. Check the unique airline names with `flights["Airline"].unique()`. Notice anything that might need cleaning later?

A2. Total scheduled sectors over time

Following the matplotlib workflow from lecture 8A, we'll create a line plot showing how total scheduled flights changed over the years.

1. Filter to get only "All Airlines" data:

```
all_airlines = flights[flights["Airline"] == "All Airlines"]
```

2. Group by year and sum the scheduled sectors:

```
yearly_total = all_airlines.groupby("Year")["Sectors_Scheduled"]  
].sum().reset_index()
```

3. Create a figure and axes:

```
fig, ax = plt.subplots(figsize=(10, 6))
```

4. Plot the line using `ax.plot()` or `yearly_total.plot.line()`:

```
ax.plot(yearly_total["Year"], yearly_total["Sectors_Scheduled"],  
        marker="o", linewidth=2, markersize=5)
```

5. Add labels and title:

```
ax.set_xlabel("Year")  
ax.set_ylabel("Total Scheduled Sectors")  
ax.set_title("Australian Domestic Flight Sectors Over Time")  
fig.tight_layout()
```

6. What patterns do you notice? Can you identify events (e.g., 2020 pandemic, 2004 data collection start)?

A3. Verifying data consistency

You mentioned that sectors scheduled should equal arrivals on time + arrivals delayed + cancellations. Let's verify this.

1. Create a calculated total:

```
all_airlines["Calculated_Total"] = (  
    all_airlines["Arrivals_On_Time"] +  
    all_airlines["Arrivals_Delayed"] +  
    all_airlines["Cancellations"]  
)
```

2. Check for mismatches:

```
mismatch = all_airlines[  
    all_airlines["Sectors_Scheduled"] != all_airlines["Calculated_Total"]  
]  
print(f"Found {len(mismatch)} mismatches")  
print(mismatch[["Year", "Month", "Sectors_Scheduled", "Calculated_Total"]])
```

3. Which years have inconsistencies? Does this affect your interpretation?

A4. Comparing individual airlines

Now let's compare trends for specific airlines using multiple lines on one plot, as demonstrated in lecture 8B with colour-coding.

1. Pick 3–4 major airlines (e.g., “Qantas”, “Virgin Australia”, “Jetstar”, “Regional Express”):

```
airlines_to_plot = ["Qantas", "Virgin Australia", "Jetstar"]
```

2. Create a new figure:

```
fig, ax = plt.subplots(figsize=(12, 6))
```

3. For each airline, filter, group by year, and plot:

```
for airline in airlines_to_plot:
    airline_data = flights[flights["Airline"] == airline]
    yearly = airline_data.groupby("Year")[
        "Sectors_Scheduled"
    ].sum().reset_index()
    ax.plot(yearly["Year"], yearly["Sectors_Scheduled"],
            marker="o", linewidth=2, label=airline)
```

4. Add a legend using `ax.legend()`.
5. Add labels and title. Use `fig.tight_layout()`.
6. What story does this tell? Which airline grew fastest? Which declined?

Part B — Data Cleaning and Aggregation (30 min)

Real-world data is messy. Let's practice data preprocessing before visualization.

B1. Cleaning airline names

You may have noticed inconsistencies in airline names (e.g., “virgin Australia” vs “Virgin Australia”, “MacAir” vs “Macair”).

1. Standardize capitalization:

```
flights["Airline_Clean"] = flights["Airline"].str.title()
```

2. Check unique values again: `flights["Airline_Clean"].unique()`.

3. Manually fix remaining issues using `replace()`:

```
flights["Airline_Clean"] = flights["Airline_Clean"].replace({
    "Macair": "MacAir",
    # Add other fixes as needed
})
```

B2. Removing aggregate rows

The “All Airlines” rows are useful for overview, but when calculating per-airline statistics, we need to exclude them.

1. Create a filtered dataset:

```
flights_per_airline = flights[
    flights["Airline_Clean"] != "All Airlines"
].copy()
```

2. Verify the count: how many rows did you remove?

B3. Average flights per airline

Calculate the mean scheduled sectors per airline across the entire dataset.

1. Group by airline and calculate mean:

```
avg_per_airline = flights_per_airline.groupby("Airline_Clean")["Sectors_Scheduled"]\
    .mean().reset_index()
avg_per_airline = avg_per_airline.sort_values("Sectors_Scheduled", ascending=False)
)
```

2. Create a horizontal bar chart (following lecture 8C):

```
fig, ax = plt.subplots(figsize=(10, 8))
avg_per_airline.plot.barh(x="Airline_Clean",
                        y="Sectors_Scheduled",
                        ax=ax, legend=False)
ax.set_xlabel("Average Scheduled Sectors per Month")
ax.set_ylabel("Airline")
ax.set_title("Average Monthly Flight Activity by Airline")
fig.tight_layout()
```

3. Use `ax.invert_yaxis()` if you want the highest values at the top.
4. Which airlines operate the most flights on average? Which are smallest?

B4. Total flights per airline over all time

Instead of average, let's look at total contribution.

1. Calculate total sectors scheduled:

```
total_per_airline = flights_per_airline.groupby("Airline_Clean")["Sectors_Scheduled"]\
    .sum().reset_index()
total_per_airline = total_per_airline.sort_values("Sectors_Scheduled", ascending=False)
)
```

2. Create a bar chart (vertical this time):

```
fig, ax = plt.subplots(figsize=(12, 6))
total_per_airline.plot.bar(x="Airline_Clean",
                        y="Sectors_Scheduled",
                        ax=ax, legend=False, color="steelblue")
ax.set_xlabel("Airline")
ax.set_ylabel("Total Scheduled Sectors (2004-2024)")
ax.set_title("Total Flight Activity by Airline")
ax.tick_params(axis="x", rotation=45)
fig.tight_layout()
```

3. How does this differ from the average? Why might an airline have high total but low average?

Part C — Delay propagation analysis (30 min)

Let's investigate how departure delays relate to arrival delays using scatter plots and linear regression.

C1. Calculate delay rates per airline

1. For each airline, calculate departure and arrival delay rates:

```
airline_delays = flights_per_airline.groupby("Airline_Clean").agg({
    "Departures_Delayed": "sum",
    "Arrivals_Delayed": "sum",
    "Sectors_Flown": "sum"
}).reset_index()

airline_delays["Departure_Delay_Rate"] = (
    airline_delays["Departures_Delayed"] /
    airline_delays["Sectors_Flown"]
)
airline_delays["Arrival_Delay_Rate"] = (
    airline_delays["Arrivals_Delayed"] /
    airline_delays["Sectors_Flown"]
)
```

2. Examine the data: `airline_delays.head()`. What do the delay rates represent?

C2. Scatter plot with colour-coded airlines

Following lecture 8B's approach to colour-coding categories, we'll visualize delay propagation.

1. Create a color palette using matplotlib's colormaps:

```
import matplotlib.pyplot as plt
cmap = plt.cm.Set2
airlines_list = airline_delays["Airline_Clean"].unique()
palette = {airline: cmap(i) for i, airline in enumerate(airlines_list)}
```

2. Map colors to each airline:

```
airline_delays["Color"] = airline_delays["Airline_Clean"].map(palette)
```

3. Create the scatter plot:

```
fig, ax = plt.subplots(figsize=(10, 8))
ax.scatter(airline_delays["Departure_Delay_Rate"],
           airline_delays["Arrival_Delay_Rate"],
           c=airline_delays["Color"],
           s=100,
           edgecolor="white",
           linewidth=1.5,
           alpha=0.8)
ax.set_xlabel("Departure Delay Rate")
ax.set_ylabel("Arrival Delay Rate")
ax.set_title("Delay Propagation: Departure vs Arrival Delays by Airline")
fig.tight_layout()
```

4. What pattern do you observe? Do airlines with high departure delays also have high arrival delays?

C3. Add a linear trendline

Now apply lecture 8C's linear regression approach to model the relationship.

1. Fit a linear model:

```

from sklearn.linear_model import LinearRegression

X = airline_delays[["Departure_Delay_Rate"]].values
y = airline_delays[["Arrival_Delay_Rate"]].values

model = LinearRegression()
model.fit(X, y)

# Generate points for the trendline
x_range = np.linspace(X.min(), X.max(), 100)
y_pred = model.predict(x_range.reshape(-1, 1))

```

2. Add the trendline to your existing plot:

```

ax.plot(x_range, y_pred,
        color="red",
        linewidth=2,
        linestyle="--",
        label=f"y = {model.coef_[0]:.2f}x + {model.intercept_:.3f}")
ax.legend()

```

3. What does the slope tell you? If the slope is close to 1.0, what does that mean about delay propagation?
4. Add a diagonal reference line (`ax.plot([0, 0.5], [0, 0.5], 'k:', alpha=0.3)`) to show where departure delay equals arrival delay.

C4. Add airline labels (optional)

1. Annotate a few airlines to identify outliers:

```

for idx, row in airline_delays.iterrows():
    if (row["Departure_Delay_Rate"] > 0.3 or
        row["Arrival_Delay_Rate"] > 0.3):
        ax.annotate(row["Airline_Clean"],
                    (row["Departure_Delay_Rate"],
                     row["Arrival_Delay_Rate"]),
                    fontsize=8,
                    alpha=0.7)

```

2. Which airlines are outliers? Are any better at recovering from departure delays?

Stretch goals

- Create a line plot showing on-time performance trends over the years. Has punctuality improved or worsened?
- Use `plt.subplots(2, 1)` to create a two-panel figure: top panel shows total flights over time, bottom panel shows average on-time percentage over time.
- Color-code your line plots using a palette from `matplotlib.cm` (lecture 8B).
- Investigate specific routes: which city pairs have the most flights? Create a bar chart of top 10 routes.
- Save your figures in both PNG and PDF formats using `fig.savefig()`.

Quiz exercises

Once you've completed the practical exercises above, work through the Week 8 quiz questions on iLearn. This is the first week where there are CodeRunner exercises. Ask your demonstrator for help if you have not done exercises like these before.